

ConstraintLayout no Android

Professor: Henrique Galati
IFSP - Campus Araraquara
Análise e Desenvolvimento de Sistemas

Ao criar interfaces visuais no Android, desenvolvedores iniciantes tendem a usar **LinearLayout** por sua simplicidade. No entanto, conforme o layout da tela se torna mais complexo, a necessidade de alinhar elementos em diferentes direções e camadas gera muitos **layouts aninhados**. Isso degrada a **performance**, **dificulta manutenção** e torna o código mais confuso.

Exemplo:

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <TextView
        android:id="@+id/txtTitulo"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Login"
        android:textSize="24sp"
        android:layout_gravity="center_horizontal"
        android:layout_marginBottom="24dp" />

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:layout_marginBottom="8dp">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Email:"
            android:layout_gravity="center_vertical" />

        <EditText
            android:id="@+id/editEmail"
            android:layout_width="0dp"
            android:layout_weight="1"
            android:layout_height="wrap_content"
            android:inputType="textEmailAddress" />
    </LinearLayout>

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
```

```

        android:layout_marginBottom="16dp">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Senha:"
            android:layout_gravity="center_vertical" />

        <EditText
            android:id="@+id/editSenha"
            android:layout_width="0dp"
            android:layout_weight="1"
            android:layout_height="wrap_content"
            android:inputType="textPassword" />
    </LinearLayout>

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:gravity="center_horizontal">

        <Button
            android:id="@+id/btnEntrar"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Entrar" />

        <Button
            android:id="@+id/btnCancelar"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Cancelar"
            android:layout_marginStart="16dp" />
    </LinearLayout>
</LinearLayout>

```

O **ConstraintLayout** surge como uma solução moderna, eficiente e flexível para interfaces responsivas. Ele permite que todos os elementos sejam posicionados **em relação a outros elementos** ou ao layout pai, eliminando a necessidade de estruturas hierárquicas complexas.

Exemplo acima convertido em **ConstraintLayout**:

```

<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="16dp">

    <TextView
        android:id="@+id/txtTitulo"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Login"
        android:textSize="24sp"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent"

```

```

        app:layout_constraintEnd_toEndOf="parent" />

<TextView
    android:id="@+id/labelEmail"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Email:"
    app:layout_constraintTop_toBottomOf="@id/txtTitulo"
    app:layout_constraintStart_toStartOf="parent"
    android:layout_marginTop="24dp" />

<EditText
    android:id="@+id/editEmail"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:inputType="textEmailAddress"
    app:layout_constraintTop_toTopOf="@id/labelEmail"
    app:layout_constraintStart_toEndOf="@id/labelEmail"
    app:layout_constraintEnd_toEndOf="parent"
    android:layout_marginStart="8dp" />

<TextView
    android:id="@+id/labelSenha"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Senha:"
    app:layout_constraintTop_toBottomOf="@id/editEmail"
    app:layout_constraintStart_toStartOf="parent"
    android:layout_marginTop="16dp" />

<EditText
    android:id="@+id/editSenha"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:inputType="textPassword"
    app:layout_constraintTop_toTopOf="@id/labelSenha"
    app:layout_constraintStart_toEndOf="@id/labelSenha"
    app:layout_constraintEnd_toEndOf="parent"
    android:layout_marginStart="8dp" />

<Button
    android:id="@+id/btnEntrar"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Entrar"
    app:layout_constraintTop_toBottomOf="@id/editSenha"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toStartOf="@id/btnCancelar"
    android:layout_marginTop="24dp" />

<Button
    android:id="@+id/btnCancelar"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Cancelar"
    app:layout_constraintTop_toTopOf="@id/btnEntrar"
    app:layout_constraintStart_toEndOf="@id/btnEntrar"
    app:layout_constraintEnd_toEndOf="parent"
    android:layout_marginStart="16dp" />
</androidx.constraintlayout.widget.ConstraintLayout>

```

Como funciona o posicionamento no **ConstraintLayout**

Cada **View** (componente da interface) deve ter **duas constraints (restrições)** mínimas:

- Uma horizontal (start/end)
- Uma vertical (top/bottom)

Essas "restrições" dizem ao Android **onde posicionar o componente** com relação a outro componente ou ao **parent** (o próprio **ConstraintLayout**).

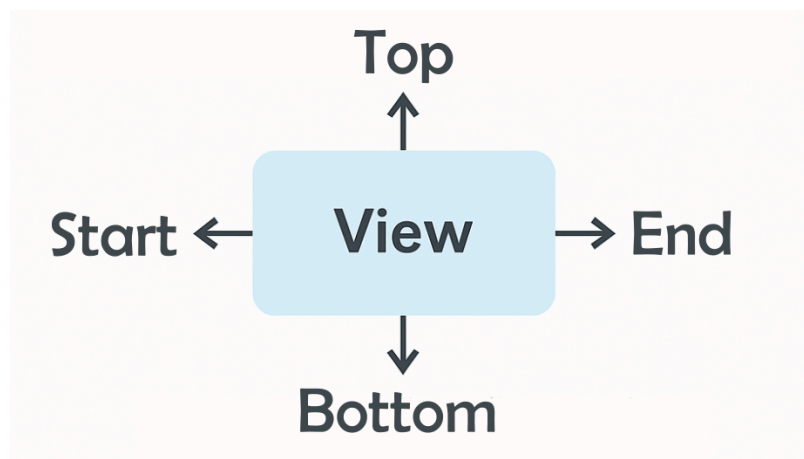


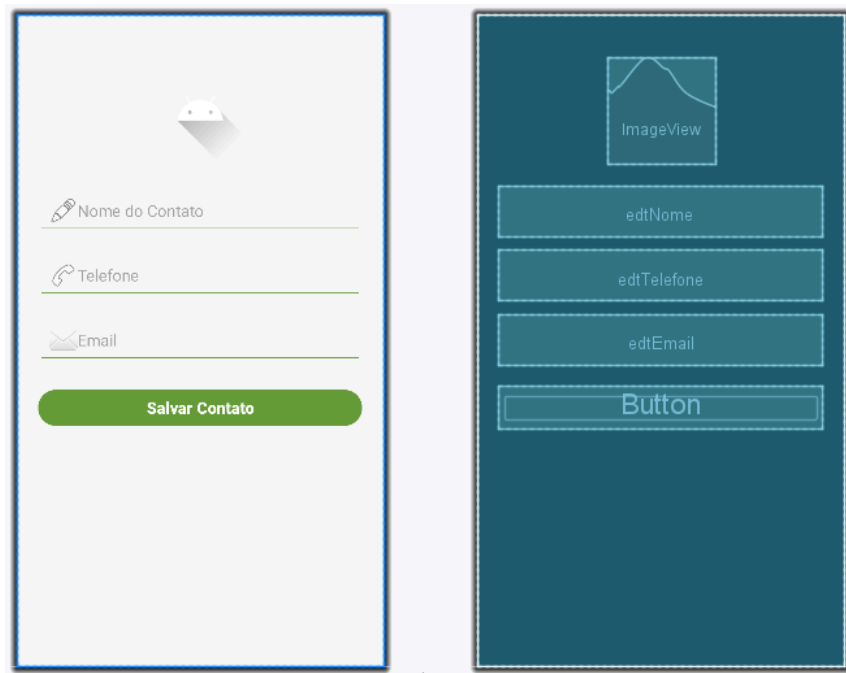
Tabela de Atributos do **ConstraintLayout**

Atributo	Direção	Descrição	Quando usar
<code>layout_constraintStart_toStartOf</code>	Horizontal	Liga a borda inicial esquerda (start) da view à borda inicial de outra view ou do parent .	Para alinhar à esquerda de outro elemento, mantendo o texto adaptado à direção (LTR/RTL).
<code>layout_constraintStart_toEndOf</code>	Horizontal	Liga a borda inicial (start) da view ao final (end) de outra.	Para posicionar um item imediatamente após outro (ex: texto ao lado de um ícone).
<code>layout_constraintEnd_toEndOf</code>	Horizontal	Liga a borda final (end) da view à borda final de outra view ou do parent .	Para alinhar a parte direita da view com outra view ou com a borda direita da tela.

<code>layout_constraintEnd_toStartOf</code>	Horizontal	Liga a borda final (end) da view ao início (start) de outra.	Para posicionar algo antes de outra view (ex: botão à esquerda de outro botão).
<code>layout_constraintTop_toTopOf</code>	Vertical	Liga o topo da view ao topo de outra view ou do <code>parent</code> .	Para alinhar a parte superior de um componente com outra view ou com o topo da tela.
<code>layout_constraintTop_toBottomOf</code>	Vertical	Liga o topo da view à parte inferior de outra.	Para colocar um elemento abaixo de outro. Muito comum em empilhamento vertical.
<code>layout_constraintBottom_toBottomOf</code>	Vertical	Liga a base da view à base de outra view ou do <code>parent</code> .	Para alinhar parte inferior com outra view ou com o final da tela.
<code>layout_constraintBottom_toTopOf</code>	Vertical	Liga a parte inferior da view ao topo de outra.	Para colocar um elemento acima de outro.

Exercício de Fixação – Cadastro de Contato (AgendaApp)

Crie uma nova tela no aplicativo **AgendaApp** para o **cadastro de contatos**. A tela deverá ser desenvolvida utilizando exclusivamente o `ConstraintLayout` como layout raiz, aplicando os conhecimentos de constraints, bias e posicionamento de elementos. A nova tela deverá conter: Título (“Novo Contato”), Foto (use uma imagem padrão), Nome, Telefone, Email e o botão Salvar. Além disso, refatore os outros layouts usando o `ConstraintLayout`. Para chamar esta nova Activity, adicione o botão na MainActivity.



Dica: para adicionar um drawable no início do `EditText`, adicione a seguinte propriedade:

```
android:drawableStart="@android:drawable/ic_menu_edit"
```

Ciclo de Vida da Activity com Intents

Nesta seção, vamos entender como o ciclo de vida da `Activity` é afetado ao navegar entre telas com `Intent` e o que muda quando utilizamos o método `finish()` para encerrar a Activity atual.

O que acontece ao abrir uma nova Activity?

Quando chamamos uma nova tela com `startActivity(intent)`, a Activity atual **não é encerrada automaticamente**. Em vez disso:

- A nova tela (Activity) é **empilhada** sobre a anterior;
- A Activity anterior entra no estado `onPause()` e, se estiver totalmente coberta, passa para `onStop()`;
- Ao pressionar o botão **Voltar**, o Android **retorna** à tela anterior, reativando seu ciclo de vida (`onRestart()` → `onStart()` → `onResume()`).

O que faz o método `finish()`?

O método `finish()` **encerra** a Activity atual e **a remove da pilha de execução**. Ele é comumente usado em casos como:

- Tela de login (que não deve voltar após o login);
- Telas que podem ser destruídas e desempilhadas.

Ao usar `finish()`, a Activity entra em `onDestroy()` e é finalizada.